

# Contents

|                                                                           |          |
|---------------------------------------------------------------------------|----------|
| <b>Guide 2 — The Recommendation Engine (simple, and how to defend it)</b> | <b>1</b> |
| 1. The idea, in three sentences . . . . .                                 | 1        |
| 2. The approach, and why . . . . .                                        | 1        |
| 3. How it works — the steps . . . . .                                     | 1        |
| 4. How a plan gets ranked (the simple version) . . . . .                  | 1        |
| 5. The files (one line each) . . . . .                                    | 2        |
| A note on the “%” (a separate, minor number) . . . . .                    | 2        |
| 6. Jury questions — ready answers . . . . .                               | 3        |

## Guide 2 — The Recommendation Engine (simple, and how to defend it)

A short, plain guide to how Mobile Match Algeria recommends plans: the idea, the approach and why, how it works, the formula on a real example, the files, and ready answers for the jury.

---

### 1. The idea, in three sentences

The user tells us their **budget**, how much **data** they need, optionally their preferred operator / type / network, and **what matters most** to them (price or data). The engine throws away every plan that doesn't fit the hard limits, scores the rest on how well they match, and returns the **top 3**, ranked. It uses no AI and no other users' data, only the one profile the user gives us.

---

### 2. The approach, and why

We use **content-based filtering with a tiered ranking** — not collaborative filtering and not AI. Why:

- A brand-new app has **no history** of what users clicked or bought, so an AI or a “people like you also chose...” model has nothing to learn from. This is the classic **cold-start problem**.
- Our catalogue is **small (~100 plans)** and every plan is fully described by numbers (price, data, minutes, SMS). Matching those numbers against the user's stated needs is enough, and it is instant.

This is a known, cited technique (the lexicographic semiorder — Fishburn 1974, Luce 1956, Tversky 1969; still used today, e.g. Safarzadeh & Rasti-Barzoki 2018), not something we invented.

**In one line:** *filter out what doesn't fit, score what's left against the user's own needs, then rank by what they said matters most.*

---

### 3. How it works — the steps

1. **Collect the profile** — budget, data need, optional operator / type / network, and 1–2 priorities ranked (price, data).
2. **Hard filters** — drop any plan over budget, or with the wrong operator / type / network. (Budget is a ceiling: a plan you can't afford is never shown.)
3. **Score two merits** (each from 0 to 1) on every surviving plan:
  - **price merit** — how cheap it is versus the budget (cheaper → closer to 1)
  - **data merit** — how well its data covers your need (covers it → 1)

4. **Tier and rank** — group plans into 5 bands by your #1 priority, then order inside each band by your #2 priority.
  5. **Return the top 3**, each with the **savings** it leaves versus your budget.
- 

#### 4. How a plan gets ranked (the simple version)

Think of it like **sorting a list by two columns** — the way a spreadsheet sorts “by Price, then by Data.” Three small steps.

**Step 1 — give each plan two scores out of 1 (we call them “merits”).** A merit just says *how good is this plan on one thing, from 0 (bad) to 1 (perfect)*: - **Price merit** = how cheap it is for your budget →  $1 - \text{price} \div \text{budget}$ . With a 2000 DA budget: a 600 DA plan scores  $1 - 600/2000 = 0.70$ ; a 1000 DA plan scores  $0.50$ . - **Data merit** = how well its data covers your need →  $\text{data} \div \text{need}$  (capped at 1). With a 10 GB need: a 4 GB plan scores  $0.40$ ; a 15 GB plan scores  $1.0$  (more than enough).

**Step 2 — drop each plan into one of 5 price “bands”.** We round the price merit to the nearest 0.25, so every plan lands on a rung: **0, 0.25, 0.5, 0.75, or 1**. Plans on the same rung are treated as **“about equally cheap.”** (*Why bother? So a 1 DA price difference doesn’t decide everything. Plans that are close on price share a band, and then data is allowed to break the tie.*)

**Step 3 — sort, exactly like a dictionary.** - First by **band** (your #1 priority, price). A cheaper band **always** beats a pricier band. - Only when two plans are in the **same band** do we look at your **#2 priority** (data) to decide who goes first.

That’s the whole thing — no secret formula, no magic number: *“sort by band, then by the tie-breaker.”*

**See it on 3 real plans** — budget 2000 DA, need 10 GB, you ranked **price #1, data #2**:

| Plan | Price   | Data  | Price merit → band      | Data merit |
|------|---------|-------|-------------------------|------------|
| C    | 600 DA  | 4 GB  | 0.70 → <b>band 0.75</b> | 0.40       |
| A    | 1000 DA | 8 GB  | 0.50 → <b>band 0.50</b> | 0.80       |
| B    | 1100 DA | 15 GB | 0.45 → <b>band 0.50</b> | 1.00       |

- **C is alone in the top band (0.75)** → C is **#1**, even though it has the *least* data. You said price matters most, so the cheapest band wins.
- **A and B are both in band 0.50** (about equally priced) → now **data** decides: B (15 GB) beats A (8 GB).
- **Final order: C → B → A.**

The one idea to remember: **your #2 priority can only shuffle plans *inside* a band — it can never lift a plan into a better band.** That’s how “price matters most” is guaranteed. (*If you pick only one priority, there’s nothing to tie-break, so we just sort by that one merit directly.*)

---

#### 5. The files (one line each)

- [src/lib/recommendation.ts](#) — the engine: the hard filters, the two merits, the tier formula, and the top-N. The whole formula above lives here.
- [src/app/api/recommendations/route.ts](#) — the API the page calls; it validates the incoming profile and is rate-limited (60 requests/min).

- [src/app/recommend/page.tsx](#) — the screen: the profile form and the ranked result cards (each showing its savings).
- [src/app/profile/page.tsx](#) — where a logged-in user saves their profile, so the daily job can recommend for them automatically.

### A note on the “%” (a separate, minor number)

The band-sort above **is** the recommendation. Separately, each plan also gets a rough **0–100 “match score”** — a quick *how-good-overall* number made by **adding up points**: budget 25, data 25, voice 15, value 15, and small amounts for SMS, features, validity, and network. Two honest things about it:

- The **method** of adding weighted points is a standard one, called **Simple Additive Weighting (SAW)**.
- The **point values** (25, 25, 15, ...) are **our own sensible choices**, not from a paper.

And it barely matters: this score does **not** decide the ranking (the band-sort does). It is used only as a **last-resort tie-break** and as the “%” in the daily “**Match found!**” notification. The cards on screen show the **rank** and the **savings**, not a %.

---

## 6. Jury questions — ready answers

**Q — Why not AI or machine learning?** AI recommenders learn from large amounts of past behaviour (what thousands of users clicked or bought). A new app has none of that — the cold-start problem. With nothing to learn from, a model would do no better than guessing, so we use the data we actually have: the user’s own stated profile.

**Q — Why content-based and not collaborative filtering, like Netflix?** Collaborative filtering means “people like you also chose X” — it needs a history of many users’ choices, which we don’t have at launch. Content-based filtering matches the plan’s own attributes (price, data...) to the user’s needs, which works from day one. The literature confirms content-based is the standard answer to cold-start.

**Q — Why tiers instead of one weighted score (e.g.  $0.6 \cdot \text{price} + 0.4 \cdot \text{data}$ )?** We tried a weighted sum first and it failed: it lets a great data score **hide** a bad price score, so a plan the user can barely afford could outrank a cheaper one — which contradicts “price matters most to me”. Tiers fix that: the #1 priority decides the band, and the #2 can only reorder inside a band. We rejected five designs before this one (documented in Chapter 3, §3.4).

**Q — Did you invent this formula, or is it researched?** The **method and the formula’s structure are researched and cited**; only the two **parameters are tuned by us** — and that is normal, not a weakness. The method is the **lexicographic semiorder**: a lexicographic order (Fishburn 1974) combined with an indifference threshold — the *semiorder* of Luce (1956), introduced as a choice model by Tversky (1969), and still used today in multi-attribute decision making (Safarzadeh & Rasti-Barzoki 2018; Taherdoost & Madanchian 2023). The ranking is a plain lexicographic two-key sort, with **no invented constant**. What is ours: the **5-tier threshold** and the value functions’ **reference points** (budget, need), tuned to our catalogue. Honest line for the jury: “*the technique is established; applying and tuning it to mobile plans is our contribution*” — never “*nothing is invented*.”

**Q — Why 5 tiers (the threshold)?** Five bands (rounding to the nearest 0.25) is enough to separate cheap / medium / expensive without splitting hairs on ~100 plans. We tried a finer step (0.1 → 11 bands) and it broke: each band held about one plan, so the 2nd priority never got to act. The number of bands is the semiorder’s “just-noticeable difference” — a tuned parameter, justified by the catalogue size. There is no secondary *weight* any more: the ranking is an explicit two-key sort, so the 2nd criterion is consulted only when two plans already share a tier.

**Q — Why this method and not TOPSIS, AHP, a weighted sum, or collaborative filtering?** Each was ruled out for a concrete reason. **Collaborative filtering / deep learning** need a click history we don't have at launch (cold-start). **Weighted sum, weighted product, and TOPSIS** are *compensatory* — great data can offset a bad price, which breaks “price matters most”. **AHP** needs numeric weights or many pairwise comparisons that ordinary users can't give. The **lexicographic semiorder** needs only a *ranking* of the criteria, is strictly non-compensatory, works with no history, and is fully transparent — the best fit for a cold-start, two-criterion, ~100-plan problem.

**Q — What if no plan fits the budget or filters?** Budget is a hard ceiling, so if nothing fits, the list is simply empty and the screen says so. We never show a plan the user can't afford; they can raise the budget or relax a filter.

**Q — Why only the top 3?** The goal is a decision, not a catalogue. Three ranked choices is enough to compare without overwhelming; the full list is always on the Offers page.

**Q — Can a user game or trick it?** There is nothing to game — the result is a deterministic function of the user's own profile and the live catalogue. No clicks, no popularity, no ads influence it.

**Q — Is this a real algorithm or just sorting?** It is an applied form of the **lexicographic semiorder**, a non-compensatory multi-criteria decision method from operations research (Fishburn 1974; Luce 1956; Tversky 1969). The “order by tier, then break ties within a tier” rule **is** that method; the sort is only the final step.

**Q — How is “savings” computed?** Simply budget – plan price (never negative). It shows how much of the monthly budget the plan leaves unspent, which is also why a cheaper plan in the same tier is preferred.

**Q — Why is budget a hard limit but data only a score?** Money is a real constraint — a plan over budget is useless, so it is filtered out entirely. Data is a preference — a plan slightly under your stated need may still be worth showing, so it is scored, not filtered.

---

*Next guide: the search engine (the token parser + FTS5). Tell me when you want it.*